- Partitioning and allocating tasks to voluntary contributors, keeping track of progress on the respective subtasks, and ensuring that everybody gets a fair chance at contributing without conflicting submissions for the same subproblems.
- Homogenising the mathematical content generated across multiple discussions spanning various forums into a coherent piece of work.
- Tracking progress relative to the goals of the project.
- Verifying the correctness of contributions made by more than fifty people with diverse backgrounds who might not share a common mathematical vocabulary, and who collaborate across multiple timezones, using a diverse set of tools.

Of the challenges mentioned above, this section deals with the first, second, and last. We briefly address the third challenge of tracking progress, the tools for which are described in Section 12. We spend a lot of time on the last point of trust and verification of results for two reasons. On the one hand, use of tools like *Lean* is fairly new in mathematical research, and while the community researching theorem provers is familiar with their guarantees and limitations, a clear academic exposition targeted at mathematics researchers will be a helpful resource for future reference, to fill a gap that is currently covered by online forums and folklore. We also describe the important role played by a number of other tools in the project.

4.2. **The Blueprint tool.** The formalization of proofs is an act of careful engineering. It is therefore helpful to have a blueprint with detailed natural language lemmata, definitions, and proof sketches in *Lean*. In the *Lean* community it has been conventional to use the *Lean blueprint* tool by Patrick Massot et al. [42]. The typical formalization project has a clearly defined set of target theorems, and the authors of the project work with a known proof, to produce a clear roadmap for the formalization. The *Lean blueprint* tool is capable of linking each piece of this natural language document to its *Lean* encoding, tracking the dependency of definitions and theorems, and progress through them, by producing a key coloured dependency graph. Thus the managers of the formalization project can not only organise the project to distribute tasks among contributors, but also track when various pieces of the formalization are complete.

In this project, we were entering uncharted mathematical territory. We had a clear list of tasks to accomplish, namely to prove the implication or anti-implication between every pair of equational laws, up to transitivity and duality. At the same time there was no clearly known pen and paper proof available for any of these beforehand. This meant that we could not prepare the blueprint of the project in advance and organise the formalization around it. Thus the traditional roles played by the blueprint were replaced by a number of other tools and mechanisms. In particular, the dependency graph did not play its traditional role in formalization projects. We developed a number of visual tools to track our progress in the project in terms of remaining open implications and anti-implications (see Section 12). Within *Lean*, every equational result was tagged with the `@[equational_result]` attribute to identify the theorem as one of the project goals, and this attribute was used to collect the status of all the goal theorems of the project. Instead of covering the dependency graph node by node, progress in the project happened as various contributors uncovered some structural ideas or heuristics that helped ATPs solve one or more pairs of laws.

The blueprint tool played a very important role in recording our progress and formalizing these classes of implications or anti-implications. It is the only comprehensive record of all the techniques that were employed in the project. Further at the level of specific implications and anti-implications, the blueprint and formalization evolved as in other projects, hand in hand. As an example, the formalization of the anti-implication E1729 $\not\models$ E817 proceeded through several iterations of refinement of the blueprint and formalization.

In conclusion, when using ITPs for tackling open problems, especially at scale, we observed that the role of the blueprint changed, but it still remained an important way to track and document our progress at a local level across the project.

4.3. **The project template.** When working on a formalization project, there are many moving pieces that need to work in concert. At the core level, there is the project set up by *Lean*'s build and dependency management system *lake*. But in addition to that, there are several pieces, including the aforementioned blueprint tool, as well as scripts that a user may choose to run to visualise various aspects of the project, or check the project in specific ways, or compile documentation automatically as the project advances. These additional tasks are accomplished by a number of external tools, and combining them in a mutually compatible way can be challenging. We side-stepped most of these issues by using the GitHub template repository of Pietro Monticone [52]. At the same time, when we began the project, the template in place was suited for more conventional formalization projects and the tooling they required. It also did not include the scripts that enabled automated project management support that we added, as well as support for deploying our visualisation tools and the paper. Over the course of the project, the `leanproject` template in turn received substantial new additions. One elementary example is the addition of git pre-push hooks, which are scripts that perform a basic sanity check on the local working copy of a contributor before pushing their contributions to the central GitHub repository.

4.4. **The *Lean* Zulip chat forum.** The *Lean* community traditionally congregates on the leanprover Zulip chat forum[6]. Our project was coordinated and organised primarily from this forum. At the beginning we created a channel called `Equational`. Zulip allows the creation and management of discussion topics within the scope of a channel. We made extensive use of the Zulip channel for several purposes. In the beginning it became the gathering point for new contributors. The new contributions process was designed and discussed on this forum. Later, topics were created for each specific technical topic, including the metatheory and its formalization, specific design decisions, specific implications and anti-implications, design of tools, etc. As shown in Figure 7, the Zulip chat served as the beginning of the contributions process for each piece of the project. Contributors first discussed their proposed contributions or specific problems they tackled on Zulip before following the steps of claiming tasks on GitHub, writing a blueprint write up and/or formalization.

4.5. **Organising the collaboration: the precedent set by the PFR project.** When five people collaborate in person, splitting up the research on a question into subtasks and assigning them to collaborators can be accomplished by discussion and consensus. When

[6] `https://leanprover.zulipchat.com`

there are more than fifty collaborators working together online, a more systematic approach is required. In previous formalization projects such as the formalization of the proof of the Polynomial Freiman–Ruzsa (PFR) conjecture [22], tasks were managed over the *Lean* zulipchat forum. The organiser of the project, Terence Tao, posted a series of message threads. Each thread corresponded to a list of outstanding tasks. These tasks were then claimed by collaborators on Zulip. The claims were recorded on a first-come first-served basis by the organiser by tagging the respective users against the tasks. Contributors could claim any open task and disclaim tasks if they couldn't finish it, with the organiser keeping track of these requests. This system allowed contributors to take their time to flesh out their work, without worrying about competing claims to the same task. Further, it helped the organisers track the task assignment and communicate with the respective collaborators to track and ascertain progress. Unfortunately, this involved a lot of manual and time-consuming management of the task list by organisers. In this project, we automated several pieces of this approach. This freed up organisers to help contributors and review their contributions.

4.6. **Organizing the collaboration in this project.** We adopted tools that are familiar to software engineers as ticket systems but are also known in the wider world of industrial production, such as the kanban system. Our project dashboard was built using the GitHub projects feature. We were able to encode some pieces of our automation using the standard GitHub-provided interface. For the rest, we relied on *continuous integration* scripts (hereon CI). The exact flow of contributions is specified in the `CONTRIBUTING.md` file of the project repository [13]. Briefly,

(1) Tasks were proposed by organisers. A contributor might start a discussion on Zulip or raise an issue on GitHub to prompt the organisers to launch tasks.
(2) Contributors could then claim tasks with a comment under the task. The CI ensured that at most one contributor could claim a task at any time.
(3) Contributors could then work on the task and propose a corresponding pull request.
(4) Upon completion of the task, the pull request received reviews, while the CI automatically checked that the project compiled and passed additional checks such as *Lean*'s environment replay tool *leanchecker* and the semi-external checker *lean4lean* [16].
(5) If all was well, the PR was merged onto the main branch of the project repository.

At any point in this process, the contributor could disclaim the task or replace a proposed PR with an alternative. In addition, organisers could always step in to fix any errors that occurred and follow up with contributors. Each of the steps described above happened automatically, triggered by a well-defined set of actions described in the `CONTRIBUTING.md` file. The typical workflow of this process is shown in the flowchart in Figure 7. The figure omits error handling and situations where organisers might manually intervene. The user interface to this project management is the GitHub project dashboard, of which we include a snapshot in Figure 8

We note that our method has since been adopted by other major formalization projects including the one to formalize Fermat's Last Theorem [15].